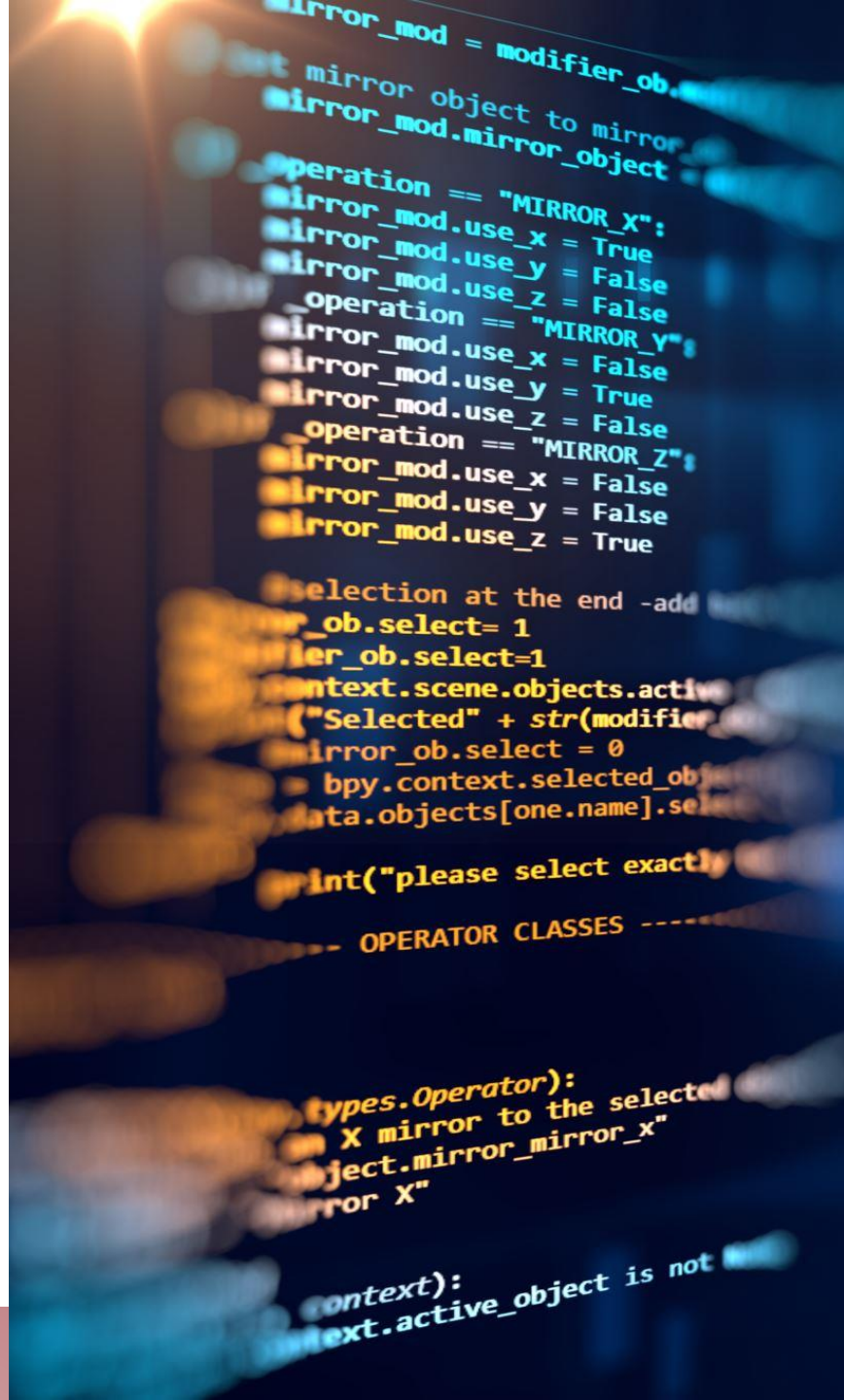
INTRODUÇÃO ÀCOMPLEXIDADE DE PROBLEMAS



COMPLEXIDADE EM TEMPO

Assumindo que o problema possa ser solucionado por algoritmos:

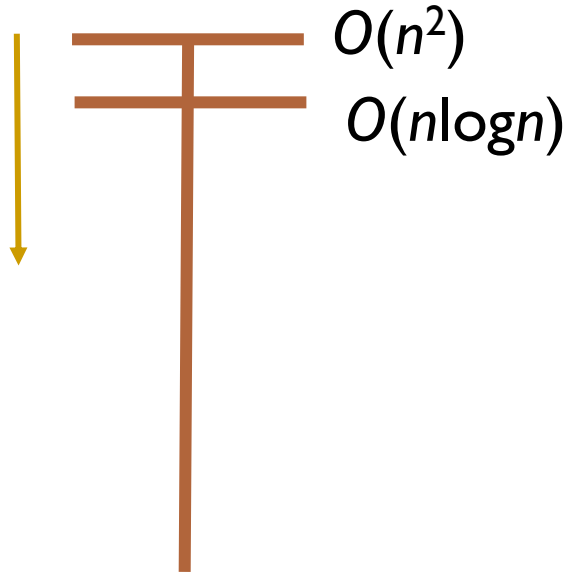
O ***LIMITE SUPERIOR*** da complexidade em tempo de um problema refere-se ao ***tempo no pior caso do melhor algoritmo*** para resolver este problema.

Exemplo: Ordenação por comparação

Mergesort – $O(n\log n)$

Quicksort – $O(n^2)$

O limite superior da complexidade da ordenação é $O(n\log n)$.

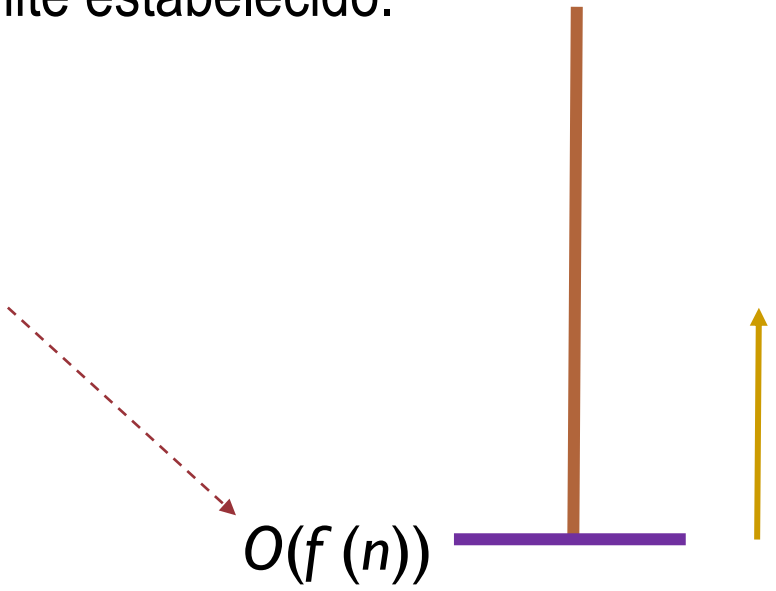


COMPLEXIDADE EM TEMPO

O ***limite inferior*** da complexidade em tempo de um problema refere-se à melhor complexidade possível.

É um resultado teórico que determina não ser possível desenvolver um algoritmo ***cuja complexidade de pior caso*** seja menor que um certo limite estabelecido.

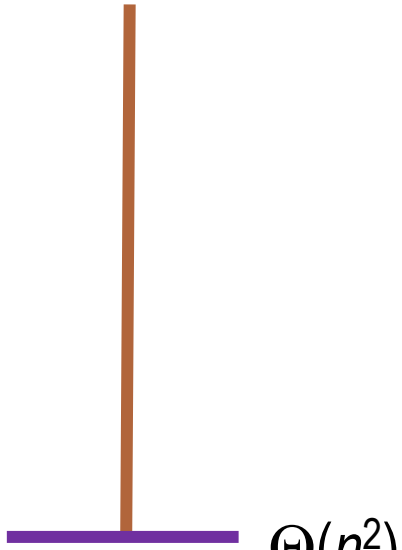
Um algoritmo para o problema terá pior caso, pelo menos



COMPLEXIDADE EM TEMPO

Exemplo: Soma e multiplicação de matrizes quadradas de ordem n.

O algoritmo precisa acessar $\Theta(n^2)$ elementos das matrizes. Portanto, um ***limite inferior*** é $\Theta(n^2)$.



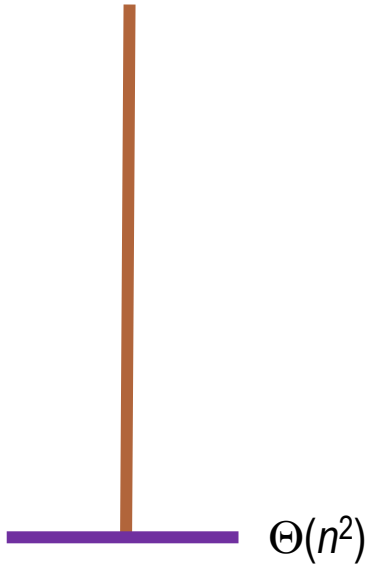
COMPLEXIDADE EM TEMPO

Exemplo: Algoritmo para soma de matrizes quadradas de ordem n.

```
Soma(A,B,C,n)
    para i ← 1 até n
        para j ← 1 até n
            C[i,j] ← A[i,j] + B[i,j]
```

Complexidade: $\Theta(n^2)$

Limite superior = Limite inferior

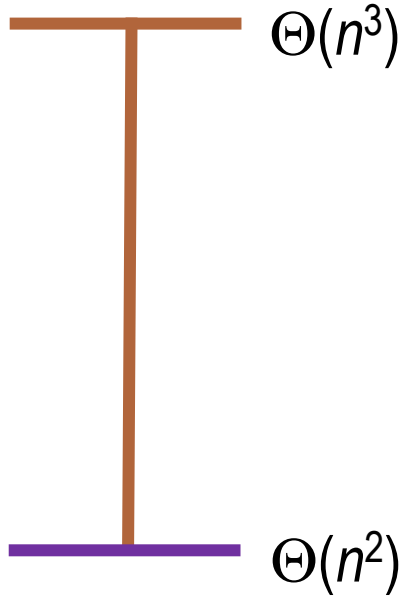


COMPLEXIDADE EM TEMPO

Exemplo: Algoritmo para multiplicação de matrizes quadradas de ordem n.

```
Soma(A,B,C,n)
    para i ← 1 até n
        para j ← 1 até n
            C[i,j] ← 0
            para k ← 1 até n
                C[i,j] ← C[i,j] + A[i,k] x B[k,j]
```

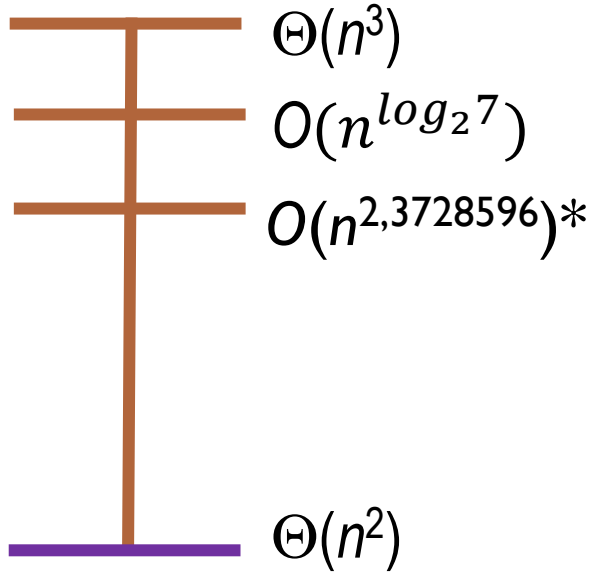
Complexidade: $\Theta(n^3)$



COMPLEXIDADE EM TEMPO

Exemplo: Algoritmo para multiplicação de matrizes quadradas de ordem *n*.

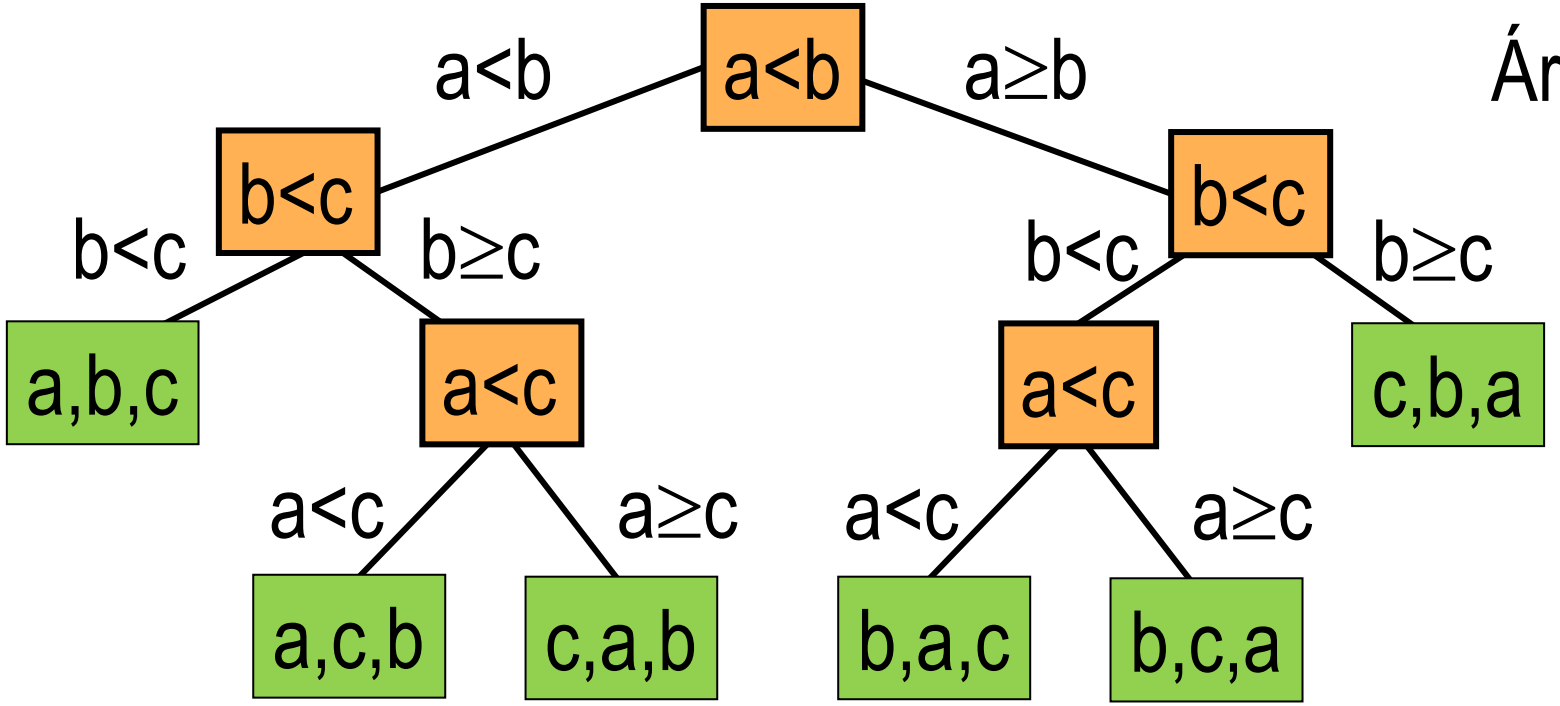
Algoritmo de Straβen: $O(n^{log_2 7}) \approx O(n^{2,807})$



** Alman, J., Williams, V. V. (2020), "A Refined Laser Method and Faster Matrix Multiplication", 32nd Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2021), arXiv:2010.05846*

COMPLEXIDADE EM TEMPO

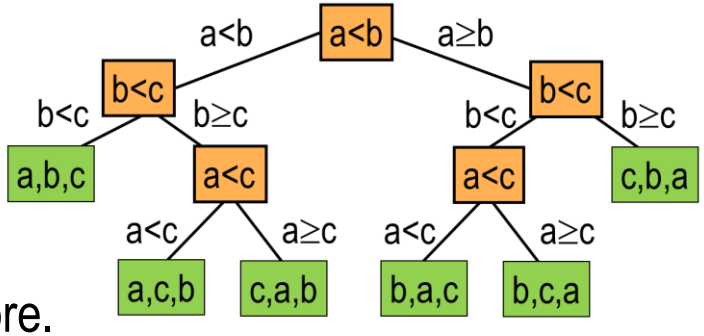
Exemplo: Ordenação por comparação - Ordenar três números inteiros a, b e c.



Árvore de Decisão

COMPLEXIDADE EM TEMPO

Exemplo: Ordenação por comparação - Ordenar três números inteiros a, b e c.



A complexidade é a profundidade de uma folha na árvore de decisão.

A **complexidade de pior caso** é a **maior profundidade** de uma folha **=** altura da árvore.

Relação entre o número de folhas e a altura de uma árvore binária completa é **#folhas = 2^{altura}**

#folhas = *n*!, onde *n* é o número de elementos a serem ordenados.

Como a árvore de decisão não é necessariamente completa temos que: $n! \leq 2^{\text{altura}}$

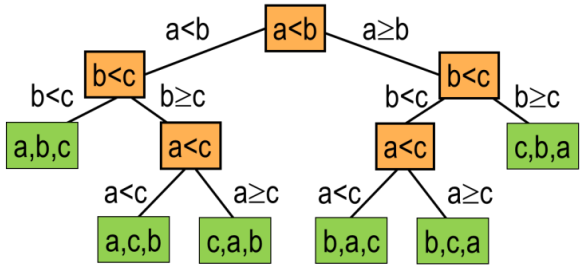
COMPLEXIDADE EM TEMPO

Exemplo: Ordenação por comparação - Ordenar três números inteiros a, b e c.

$n! \leq 2^{\text{altura}}$

$\log_2 n! \leq \log_2 2^{\text{altura}}$

$\text{altura} \geq \lceil \log_2 n! \rceil$



Aproximação de Stirling: $\lceil \log_2 n! \rceil = n \log_2 n - \frac{n}{\ln 2} + \frac{1}{2} \log_2 n + O(1)$

A complexidade de pior caso é dada pela altura da árvore. Assim: $O(n \log_2 n)$
Portanto, o limite inferior para o problema é $O(n \log n)$.

COMPLEXIDADE EM TEMPO

Existem limites inferiores naturais, como, por exemplo, o tamanho da entrada uma vez que o algoritmo deverá ler sua entrada.

Exemplo: limite inferior para soma de matrizes e para multiplicação de matrizes quadradas de ordem n é $\Omega(n^2)$.

Um limite inferior para um problema *P,* a função *l*, é tal que a complexidade de pior caso de qualquer algoritmo que resolva *P* é $\Omega(l)$.

Se existir algoritmo *A* tal que *A* é $O(l)$, então *A* é dito ***algoritmo ótimo para P***.